LangGraph RAG Extraction Pipeline — Executive Summary

Memorial Sloan Kettering | Goel Lab
Author: James Roberts

Objective: Reduce AI fabrication and omission rates in structured clinical feature extraction from deidentified breast cancer OCR reports using a multi-agent LangGraph pipeline with feature-specific RAG retrieval and evidence-grounded verification.

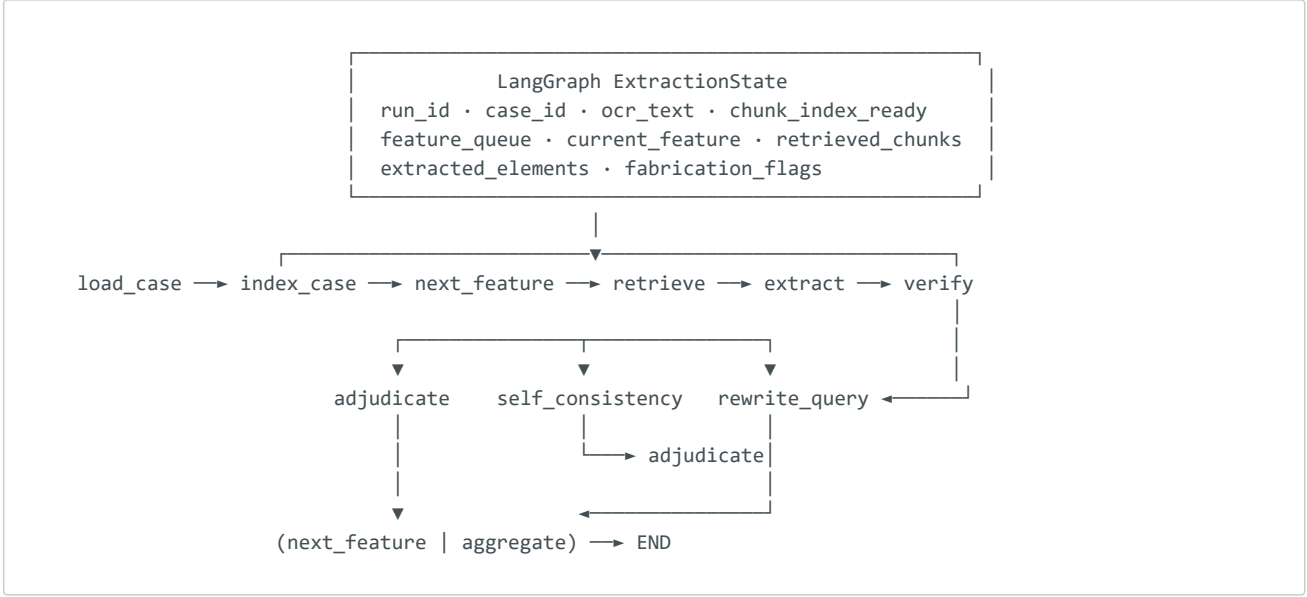
Executive Summary

This pipeline replaces single-pass LLM summarization with a stateful, multi-step agentic system. For each of 13 clinical features, the pipeline retrieves relevant text chunks, extracts a value, verifies it against source evidence, rewrites the query on failure, applies self-consistency checks on high-risk features, and adjudicates a final CORRECT / FABRICATION / OMISSION / UNCERTAIN verdict.

All outputs are auditable: every extracted value is linked to a verbatim source quote and a verification confidence score. Results are stored as structured JSON per case and aggregated into a reproducible experiment run.

The system was validated against 63 patient cases (from a 200-case validation set) where the baseline AI model produced fabrications (n=15 feature-level errors) or omissions (n=78 feature-level errors), using deidentified scanned PDFs as input.

Architecture



Routing logic (`route_after_verify`):

- `supported=True` and `confidence ≥ threshold` → **adjudicate**
- High-risk feature + `confidence < 0.8` → **self_consistency** (3 independent passes)
- `retrieval_attempts < 2` → **rewrite_query** → retrieve_again → extract → verify
- Otherwise → **adjudicate**

File Inventory

`src/workflows/` — Pipeline Orchestration

File	Purpose	Key Functions	Output
`extraction_state.py`	TypedDict schemas for all pipeline state	`make_initial_state()`, `make_empty_feature_result()`	`ExtractionState`, `FeatureResult`, `Chunk` TypedDicts
`extraction_graph.py`	Full LangGraph `StateGraph` definition	`build_extraction_graph()`, `compile_graph()`, all node functions	Compiled LangGraph app ready for `.invoke()`
`validation_graph.py`	Post-hoc comparison of pipeline output vs	`validate_batch_results()`, `summarize_validation()`, `compute_confusion()`	Merged DataFrame with classification column (TP/FP/FN/TN)

File	Purpose	Key Functions	Output
	human labels		
`orchestration.py`	Single-case and batch runner with run tracking	`run_single_case()`, `run_batch()`	Per-case JSON in `experiments/runs/{run_id}/`, aggregated parquet

`src/rag/` — Retrieval-Augmented Generation

File	Purpose	Key Functions	Output
`feature_queries.py`	Central feature registry — queries, k values, criticality, thresholds	— (data module)	`FEATURES` dict, `CRITICAL_FEATURES`, `HIGH_RISK_SELF_CONSISTENCY`, `VERIFICATION_THRESHOLD_MAP`
`embed_chunks.py`	Embed text chunks with HuggingFace sentence-transformers into FAISS	`build_faiss_index()`, `get_embedder()`, `save_faiss_index()`, `load_faiss_index()`	FAISS index object; optionally saved to disk
`vector_store.py`	In-process FAISS index cache (per-case)	`get_or_build_index()`, `clear_index()`	Cached FAISS index via `_CASE_INDEX_CACHE` dict
`retrievers.py`	Feature-specific dense retrieval returning typed `Chunk` objects	`retrieve_for_feature()`, `get_feature_query()`, `get_feature_k()`, `format_chunks_for_prompt()`	List of `Chunk` TypedDicts with `retrieval_score` populated

`src/preprocessing/` — Text Processing

File	Purpose	Key Functions	Output
`chunk_text.py`	Chunk OCR text with `RecursiveCharacterTextSplitter`; infer modality labels	`chunk_ocr_text()`, `infer_modality()`	List of `Chunk` TypedDicts with `chunk_id`, `modality`, `page_num`, `token_count`

`src/prompts/` — Prompt Engineering

File	Purpose	Key Functions	Output
`extraction_prompt_builder.py`	Build anti-fabrication extraction prompt with CoT + few-shot	`build_extraction_prompt()`	`[SystemMessage, HumanMessage]` list for LangChain LLM
`verification_prompt_builder.py`	Build RAG verification prompt to check claim support	`build_verification_prompt()`	`[SystemMessage, HumanMessage]` list
`render_prompt.py`	Load and render YAML prompt templates with variable substitution	`get_extraction_prompt_template()`, `get_verification_prompt_template()`, `get_rewrite_prompt_template()`	Rendered prompt string
`lcp_optimizer.py`	Contrastive Prompt Learning scorer and candidate generator	`score_prompt_variant()`, `rank_prompt_variants()`, `build_contrastive_summary()`, `generate_candidate_revision_prompt()`	Ranked DataFrame; contrastive summary string for LLM-based revision

`src/agents/` — LangGraph Node Functions

File	Purpose	Key Functions	Output
`extract_agent.py`	Call Claude to extract one feature from retrieved chunks	`extract_feature(state)`	Updated `ExtractionState` w `FeatureResult` in `extracted_elements[featu`
`verify_agent.py`	Call Claude to verify extraction claim against source chunks	`verify_feature(state)`	Updated `ExtractionState` w `supported`, `verification_quote`, `verification_confidence`

File	Purpose	Key Functions	Output
`rewrite_agent.py`	Call Claude to rewrite failed retrieval query with synonyms	`rewrite_query(state)`	Updated `ExtractionState` w new `current_query`
`adjudicate_agent.py`	Assign CORRECT / FABRICATION / OMISSION / UNCERTAIN verdict	`adjudicate_result(state)`	Updated `ExtractionState` w `verdict` and updated `fabrication_flags`
`self_consistency_agent.py`	Run 3 independent extraction passes on high-risk features	`self_consistency_check(state)`	Updated `ExtractionState` w agreed/indeterminate value a updated confidence

`src/graph/` — Knowledge Graph

File	Purpose	Key Functions	Output
`graph_schema.py`	Dataclass node/edge definitions for the clinical evidence graph	— (data module)	`KnowledgeGraph`, `PatientNode`, `ClinicalFeatureNode`, `EvidenceChunkNode`, etc.
`build_graph.py`	Build NetworkX DiGraph from batch results; save/load GraphML	`results_to_kg()`, `build_networkx_graph()`, `save_graphml()`, `load_graphml()`	`nx.DiGraph`; `*.graphml` file
`kg_retriever.py`	Query the knowledge graph by case, verdict, or feature	`get_features_by_case()`, `get_fabrications()`, `get_evidence_for_feature()`, `summarize_graph_stats()`	Lists of node attribute dicts
`neo4j_io.py`	Optional Neo4j export (falls back gracefully if driver absent)	`export_to_neo4j()`	Neo4j nodes and relationships (no return value)

`src/utils/` — Shared Utilities

File	Purpose	Key Functions	Output
`logging_utils.py`	Structured pipeline logging with node/feature/verdict context	`get_logger()`, `log_node_entry()`, `log_verdict()`, `log_fabrication_flag()`, `log_retrieval()`	Formatted log lines to stdout
`json_utils.py`	Safe LLM JSON parsing (handles markdown fences, truncation)	`safe_parse_json()`, `validate_feature_result()`, `validate_verification_result()`, `coerce_confidence()`, `coerce_page_refs()`	Parsed dict or `None`; coerced values
`io_utils.py`	File I/O, run directory management, result flattening	`generate_run_id()`, `save_json()`, `load_json()`, `make_run_dir()`, `save_case_result()`, `flatten_case_results_to_df()`	JSON files; parquet; run directories; flat DataFrame

`eval/schemas/` — Pydantic Validation Schemas

File	Purpose	Key Classes	Output
`feature_schema.py`	Validate and coerce LLM extraction output	`FeatureExtractionOutput`, `VerificationOutput`, `FeatureResult`	Validated Pydantic model instances
`verification_schema.py`	Verify verification output structure + confidence rubric	`VerificationResult`, `CONFIDENCE_RUBRIC`	Validated model; `.to_rubric_label()` string
`hcat_schema.py`	HCAT safety score schema (Harm, Calibration, Accuracy, Traceability)	`HCATScore`, `HCATBatchReport`	Pydantic models; `.safety_score` property; `.mean_fabrication_rate`

`eval/metrics/` — Evaluation Metrics

File	Purpose	Key Functions	Output
`fabrication_metrics.py`	**Primary safety endpoint**:	`compute_fabrication_rate()`, `fabrication_by_feature()`, `fabrication_by_prompt()`, `high_fabrication_features()`	Rate dict; ranked DataFrames

File	Purpose	Key Functions	Output
	fabrication rate		
`extraction_metrics.py`	Feature-level accuracy, F1, precision, recall vs human labels	`decode_human_labels()`, `compute_overall_metrics()`, `compute_feature_metrics()`	Metrics dict with classification report; ranked DataFrame
`retrieval_metrics.py`	RAG retrieval quality: precision@k, rewrite rate, pass rate	`precision_at_k()`, `mean_precision_at_k()`, `retrieval_summary_by_feature()`, `compute_retrieval_stats()`	Float scores; summary DataFrame
`self_consistency_metrics.py`	Self-consistency pass/fail rates for high-risk features	`sc_agreement_rate()`, `sc_by_feature()`	Summary dict; DataFrame
`hcat_metrics.py`	Full HCAT batch safety report	`compute_hcat_score()`, `compute_batch_hcat()`, `hcat_report_to_df()`	`HCATScore`; `HCATBatchReport`; flat DataFrame

`models/configs/` — Configuration YAMLs

File	Contents
`rag_config.yaml`	Chunk size, overlap, embedding model, retrieval k values, self-consistency settings
`model_registry.yaml`	Claude / GPT-4o model specs: temperature, max tokens, context window, use-cases
`safety_thresholds.yaml`	Per-feature verification thresholds, confidence rubric, adjudication rules, zero-tolerance features
`graph_config.yaml`	Knowledge graph backend, GraphML paths, Neo4j connection, node/edge type definitions

`prompts/library/` — Prompt Templates (YAML)

File	Contents
`feature_queries.yaml`	Per-feature retrieval queries and k values

File	Contents
`extraction_prompts.yaml`	`default`, `cot_few_shot`, and `rag_verify_v1` extraction prompt templates
`verification_prompts.yaml`	`default` and `strict` verification prompt templates
`rewrite_prompts.yaml`	`default` and `synonym_expand` rewrite templates
`few_shot_examples.yaml`	Curated few-shot examples for 5 key features (lesion size, invasive size, receptor status, clip, biopsy)

`prompts/frozen/` — Prompt Registry

File	Contents
`prompt_metadata.json`	Registry of 4 frozen prompt variants (P1–P4) with technique, RAG/CoT/few-shot flags, dates

`fabrication_analysis/` — Analysis Notebooks

Notebook	Purpose	Outputs
`01_langgraph_extraction_pipeline.ipynb`	Full end-to-end pipeline demo + single-case run + HCAT scoring	Verdict distribution figure, confidence by feature figure, verification stats figure, case JSON, knowledge graph GraphML
`02_document_text_metrics.ipynb`	Document and text quality visualization suite	OCR quality histograms, fabrication rate by feature/prompt/heatmap, confidence scatter, retrieval stats, SC analysis, HCAT distributions
`03_fabrication_omission_pipeline.ipynb`	Re-run LangGraph pipeline on all 63 AI error cases; compare to human labels	Recovery rate by feature, stacked bar figures, error heatmap, `audit_table.csv`, `comparison_results.csv`, `run_summary.json`

`tools/`

File	Purpose	Output
`check_fab_pipeline.py`	Sanity check: validates MRN→PDF mapping for all 63 error cases	Console report: matched rows, PDF existence, OCR cache status, feature error breakdown

Data Flow

```
Private PDFs (DATA_PRIVATE_DIR)
        │
        ▼
[pytesseract + fitz OCR]  ←── src/deidentification/ocr_quality_scoring.py
        │  ~4.6s/page, cached to ocr_cache/*.txt
        ▼
[chunk_text.py]  →  List[Chunk]  (1000-char, 200-overlap, modality-labelled)
        │
        ▼
[embed_chunks.py]  →  FAISS index per case (sentence-transformers/all-mpnet-base-v2)
        │
        ▼
[LangGraph: extraction_graph.py]
  per feature (13 features):
    retrieve → extract → verify → (adjudicate | rewrite | self_consistency)
        │
        ▼
[ExtractionState.extracted_elements]
  {feature_name: FeatureResult{value, evidence, confidence, verdict, verification_quote}}
        │
        ├──► experiments/runs/{run_id}/{case_id}.json  (per-case auditable JSON)
        ├──► experiments/runs/{run_id}/feature_outputs.parquet  (flat table)
        ├──► data/knowledge_graph/{run_id}_kg.graphml  (evidence graph)
        └──► eval/metrics/  →  HCAT safety report  →  reports/*.png
```

Key Safety Design Decisions

Decision	Rationale
Per-feature retrieval (not full-doc)	Reduces noise; improves source traceability
Verification agent	Catches fabrications the extractor missed; forces source citation
Query rewriting on failed verification	Expands retrieval to synonyms/modality context before giving up
Self-consistency on receptor status + invasive size only	Avoids 3× API cost for low-risk features; focuses on clinical zero-tolerance targets
`Not reported` **not verified**	Avoids false fabrication flags on legitimately absent features
Incremental JSON saving in batch runner	Pipeline is resumable after interruption; no lost work
Verification threshold tuning per feature	Receptor status (0.9), invasive size (0.85), others (0.7–0.8) — calibrated to clinical risk

---

Running the Pipeline

Prerequisites

```
uv add langgraph langchain-anthropic langchain-community langchain-huggingface
uv add langchain-text-splitters faiss-cpu sentence-transformers
uv add networkx pydantic pyyaml python-dotenv pyarrow
```

Environment

```
cp .env.example .env
# Set: ANTHROPIC_API_KEY, PROJECT_ROOT, DATA_PRIVATE_DIR
```

Single case

```
from src.workflows.orchestration import run_single_case
result = run_single_case(case_id="CASE_001", ocr_text="...", prompt_id="rag_verify_v1")
```

Batch (63 error cases)

Open `fabrication_analysis/03_fabrication_omission_pipeline.ipynb` and run all cells. Set `DRY_RUN = False`. OCR caches to `DATA_PRIVATE_DIR/ocr_cache/` (~43 min first run, instant thereafter).

Sanity check

```
python tools/check_fab_pipeline.py
```

Memorial Sloan Kettering Cancer Center | Goel Lab | 2025–2026